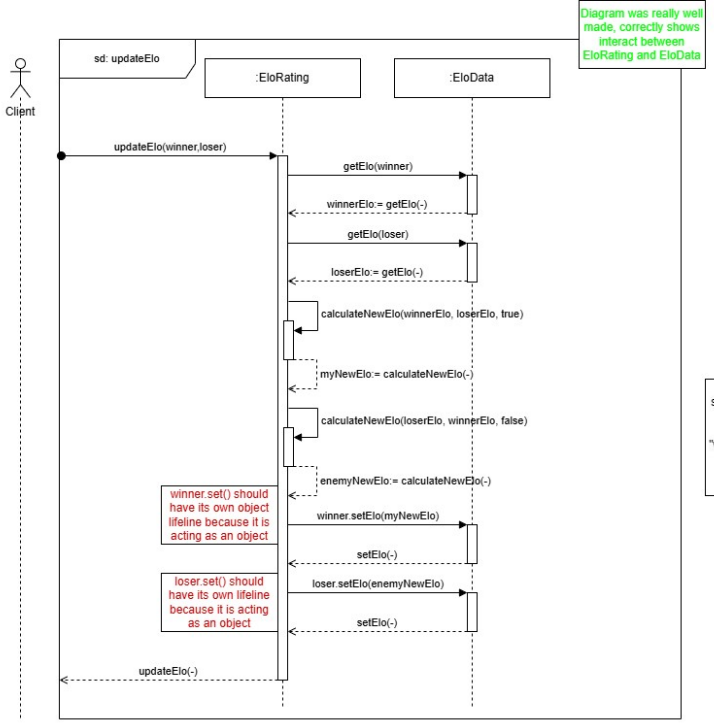
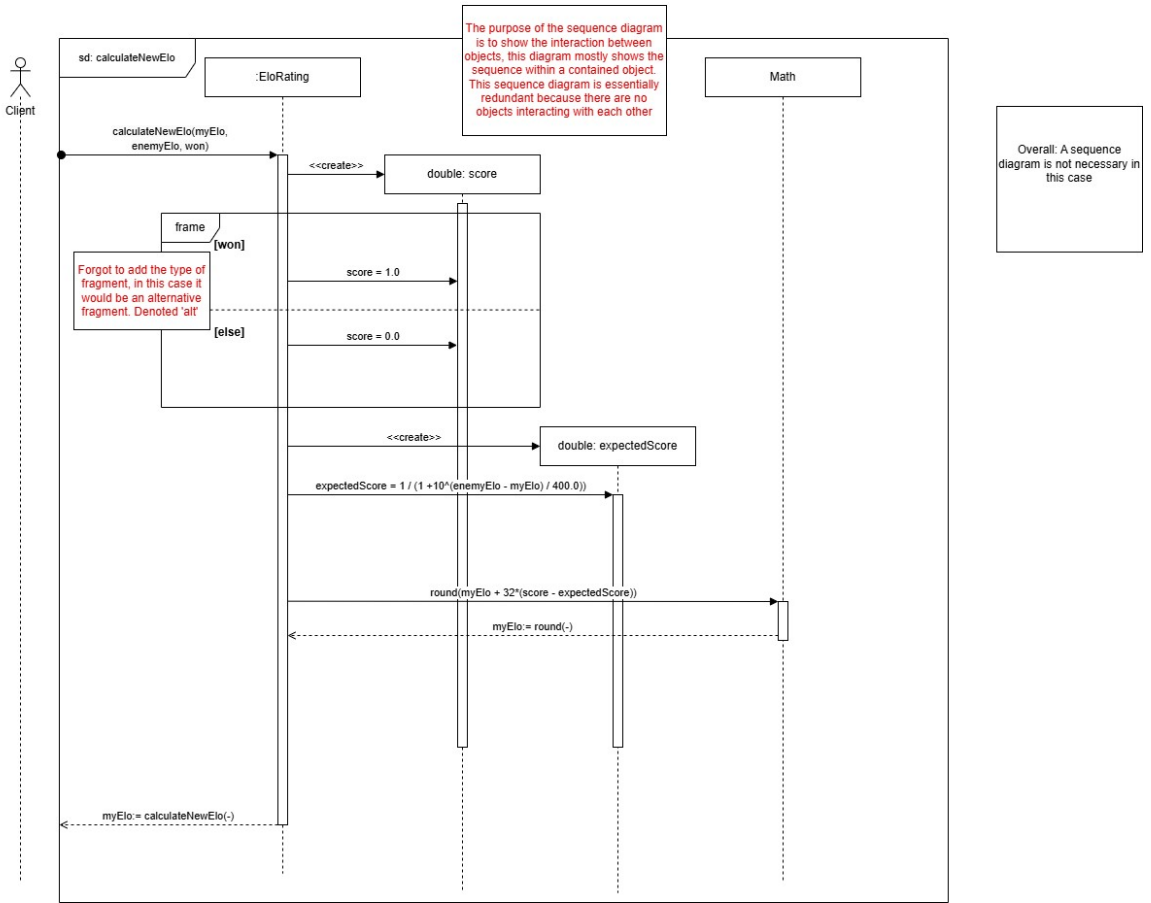

Diagram was really well made, correctly shows interact between EloRating and EloData
sd: updateElo
Client
:EloRating
:EloData
updateElo(winner,loser)
getElo(winner)
winnerElo:= getElo(-)
getElo(loser)
loserElo:= getElo(-)
calculateNewElo(winnerElo, loserElo, true)
myNewElo:= calculateNewElo(-)
calculateNewElo(loserElo, winnerElo, false)
enemyNewElo:= calculateNewElo(-)
winner.set() should have its own object lifeline because it is acting as an object
winner.setElo(myNewElo)
setElo(-)
loser.set() should have its own lifeline because it is acting as an object
loser.setElo(enemyNewElo)
setElo(-)
updateElo(-)
Overall: diagram is useful to the developers on the team and should be used as a blueprint.
suggestion: Instead of only having "EloData" itself. It can be modeled as if its branching off of player objects for example: "loser:Player" and "winner:Player" or however the player's attributes are stored lead to ":EloData".


The purpose of the sequence diagram is to show the interaction between objects, this diagram mostly shows the sequence within a contained object. This sequence diagram is essentially redundant because there are no objects interacting with each other
sd: calculateNewElo
Client
:EloRating
Math
calculateNewElo(myElo, enemyElo, won)
<<create>>
double: score
frame
[won]
Forgot to add the type of fragment, in this case it would be an alternative fragment. Denoted 'alt'
score = 1.0
[else]
score = 0.0
<<create>>
double: expectedScore
expectedScore = 1 / (1 +10^(enemyElo - myElo) / 400.0))
round(myElo + 32*(score - expectedScore))
myElo:= round(-)
myElo:= calculateNewElo(-)
Overall: A sequence diagram is not necessary in this case